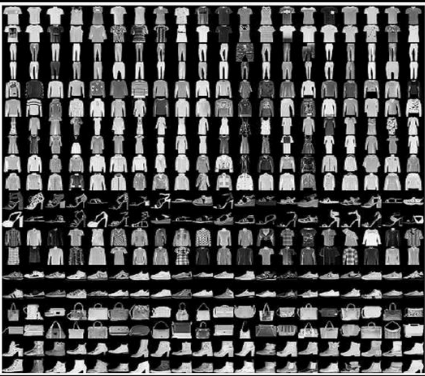


Vision par ordinateur

TP 2 – Réseaux de neurones convolutifs (Partie1)

Exercice 1– Classification d’images Fashion-MNIST avec un CNN

Label	Description	Examples
0	T-Shirt/Top	
1	Trouser	
2	Pullover	
3	Dress	
4	Coat	
5	Sandals	
6	Shirt	
7	Sneaker	
8	Bag	
9	Ankle boots	

Contenu du dataset Fashion-MNIST

- Nombre total d’images : 70 000
 - 60 000 pour l’entraînement
 - 10 000 pour le test
- Format :
 - Chaque image = matrice 28×28 (valeurs de pixels entre 0 et 255).
 - Niveau de gris (canal unique).
- Labels : 10 classes (0 à 9), représentant différents types de vêtements.

Objectif

Construire un modèle de réseau de neurones convolutionnel (CNN) pour classifier les images du dataset **Fashion-MNIST** (10 classes de vêtements). Expérimenter avec différentes architectures et analyser les résultats.

Étapes de réalisation

1. Importation des bibliothèques

- Importez TensorFlow/Keras, NumPy et Matplotlib.
- Importez le dataset **Fashion-MNIST** depuis keras.datasets.

2. Chargement et préparation des données

- Chargez les données d'entraînement et de test.
- Normalisez les pixels (valeurs entre 0 et 1).
- Redimensionnez les images pour qu'elles aient la forme (28, 28, 1).
- Convertissez les labels en *one-hot encoding*.
 - On transforme les labels en one-hot encoding.
 - Exemple : label 3 $\rightarrow$ [0,0,0,1,0,0,0,0,0,0].
- Optionnel : affichez quelques exemples d'images avec leurs labels.

3. Construction du modèle CNN

- Ajoutez une **première couche convolutionnelle** (32 filtres, 3×3, ReLU).
- Ajoutez une **seconde couche convolutionnelle** (64 filtres, 3×3, ReLU) suivie d'un **MaxPooling2D**.
- Ajoutez une couche **Dropout** pour éviter l'overfitting.
- Ajoutez une couche **Flatten**.
- Ajoutez une couche **Dense** (128 neurones, ReLU).
- Ajoutez une couche de sortie avec **10 neurones** (softmax).

4. Compilation du modèle

- Utilisez la fonction de perte categorical_crossentropy.
- Choisissez l'optimiseur Adam.
- Utilisez la métrique accuracy.

5. Entraînement du modèle

- Entraînez sur les données d'entraînement (par ex. 10 époques, batch size 64).
- Validez sur l'ensemble de test.

6. Évaluation et analyse

- Évaluez le modèle sur les données de test.
- Affichez la précision obtenue.
- Tracez les courbes **accuracy** et **loss**.
- Identifiez les classes les plus confondues grâce à une **matrice de confusion**.

Exercice 2– Classification d'images MNIST avec un CNN



Contenu du dataset MNIST

- Nombre total d'images : 70 000
 - 60 000 pour l'entraînement
 - 10 000 pour le test
- Format des images :
 - Taille : 28 × 28 pixels (soit 784 pixels par image).
 - Niveau de gris (canal unique).
 - Valeurs des pixels : 0 à 255 (intensité $\rightarrow$ noir/blanc).

Labels (classes)

- Il y a 10 classes, correspondant aux chiffres de 0 à 9.

Objectif :

Construire un CNN plus profond pour classifier les chiffres manuscrits MNIST, en expérimentant sur certaines techniques de régularisation et prétraitement, tout en analysant l'impact de quelques hyperparamètres.

Étapes à réaliser :

1. Importation des bibliothèques

- TensorFlow/Keras, NumPy, Matplotlib.
- ImageDataGenerator pour l'augmentation de données.
- BatchNormalization et Dropout pour la régularisation.

2. Chargement et préparation des données

- Chargez le dataset MNIST depuis Keras.
- Normalisez les pixels (0 à 1).
- Redimensionnez les images en (28, 28, 1).
- Convertissez les labels en one-hot encoding.
- Séparez les données en train/validation/test.

3. Data Augmentation

- Créez un ImageDataGenerator avec :
 - rotation légère (10–15°),
 - translation horizontale et verticale (0.1),
 - zoom léger (0.1).
- Expliquez que l'augmentation permet d'éviter l'overfitting et de rendre le modèle plus robuste.

4. Définition du modèle CNN

- Deux ou trois blocs convolution + pooling :
 - **Bloc 1** : Conv2D (32 filtres, 3×3, ReLU) → BatchNormalization → MaxPooling2D → Dropout (0.25)
 - **Bloc 2** : Conv2D (64 filtres, 3×3, ReLU) → BatchNormalization → MaxPooling2D → Dropout (0.25)
 - **Optionnel Bloc 3** : Conv2D (128 filtres, 3×3, ReLU) → BatchNormalization → MaxPooling2D → Dropout (0.25)
- Flatten → Dense 128 neurones (ReLU) → Dropout (0.5) → Dense sortie 10 neurones (softmax).

5. Compilation du modèle

- Loss : categorical_crossentropy.
- Optimiseur : Adam.
- Métrique : accuracy.

6. Entraînement et validation

- Entraînez le modèle 5–15 époques avec batch size 64 ou 128.
- Utilisez validation_split ou un jeu de validation séparé.
- Affichez les courbes loss et accuracy pour train/validation.

7. Évaluation sur le jeu de test

- Évaluez la précision finale sur le test set.
- Montrez quelques images mal classées pour analyser les erreurs.

8. Expérimentation (facultative)

- Testez l'effet de :
 - Dropout : 0.25 vs 0.5
 - BatchNormalization : avec ou sans
 - Nombre de filtres : 32/64 vs 64/128
- Comparez accuracy et loss sur validation/test.